

End-to-End Formalization: Compute-Efficient Structural Clipping for CISPO

1 The Problem: The $O(N^2)$ Memory Wall in Structural RLHF

Working backwards from empirical failures in reasoning tasks reveals that standard CISPO is artificially bottlenecked by **token-agnostic truncation**.

The standard objective applies a static scalar bound (ϵ_{high}) to the importance weight $r_t(\theta)$ for every token:

$$L_{\text{CISPO}}(\theta) = -\mathbb{E} \left[\text{detach}(\min(r_t(\theta), \epsilon_{\text{high}})) \cdot \hat{A}_t \cdot \log \pi_{\theta}(a_t \mid s_t) \right] \quad (1)$$

This mechanism clips the gradient of a critical reasoning token (e.g., a mathematical operator) exactly as it would a structural filler word, preventing the active policy (π_{θ}) from rapidly updating critical logical hubs.

To solve this, the clipping bound must be dynamic (ϵ_t), scaled by the token’s structural importance (attention in-degree). However, calculating exact attention saliency requires caching the full $N \times N$ attention matrix across multiple layers during the active RL rollout. In a standard PPO/CISPO training loop, backpropagation already consumes maximum VRAM. Forcing π_{θ} to maintain an $O(N^2)$ spatial complexity for attention matrices triggers immediate Out-of-Memory (OOM) errors and destroys training throughput.

2 The Solution: Decoupled & Sub-Linear Saliency

To resolve the bottleneck, we must extract structural saliency without caching $N \times N$ matrices on the active policy. We achieve this through a unified architecture that isolates computation and reduces spatial complexity.

2.1 Component A: Reference Model Offloading (The Systems Shortcut)

Instead of forcing the active policy (π_{θ}) to compute its own saliency, we offload the calculation entirely to the frozen reference model (π_{ref}). Because the foundational semantic routing of a prompt (the identification of logic gates and numbers) is structurally inherent to the sequence, the attention topology differs negligibly between π_{ref} and π_{θ} . We extract all structural metrics exclusively during the π_{ref} forward pass (used for KL-divergence), adding zero FLOPs or memory overhead to the active policy.

2.2 Component B: Sub-Linear Saliency Extraction

Within the offloaded π_{ref} pass, we utilize one of two highly efficient metrics to quantify the raw structural importance score (S_t) for each token without materializing the full attention matrix.

Path 1: The Architectural Proxy (Key-Vector L_2 Norms)

Tokens that act as structural hubs naturally develop massive Key vectors so future queries can attend to them. We intercept the materialized KV-cache of π_{ref} at the final reasoning layers (L_{top}) and compute the L_2 norm.

- **Spatial Complexity:** $O(1)$ overhead.
- **Metric:** $S_t = \|k_t^{(L_{\text{top}})}\|_2$

Path 2: The Exact Algorithmic Solution ($O(N)$ Streaming Accumulators)

Instead of storing the full matrix, we dynamically accumulate the in-degree score. At each generation step i , π_{ref} computes a $1 \times i$ attention vector $\mathbf{a}_i$. We initialize a 1D tensor $\mathbf{c}$ of size N and update the running total at each step: $\mathbf{c}_{1:i} \leftarrow \mathbf{c}_{1:i} + \mathbf{a}_i$.

- **Spatial Complexity:** $O(N)$ per head.
- **Metric:** $S_t = \mathbf{c}[t]$

3 End-to-End Implementation Formalization

The complete pipeline operates in three sequential steps during the post-training rollout.

3.1 Step 1: Forward Pass & Saliency Extraction

During the generation and KL-divergence phase, the reference model π_{ref} processes the sequence. We extract the raw structural importance score S_t for every token $t \in [1, T]$ using either the KV-norm proxy or the streaming accumulator.

3.2 Step 2: Dynamic Bound Normalization

To ensure the active policy’s variance remains theoretically bounded over the entire sequence, we cannot simply use S_t as the clipping threshold. We must normalize the raw scores to create a multiplier m_t , ensuring the mean of the dynamic bounds strictly equals the standard scalar ϵ_{high} .

We calculate the dynamic bound ϵ_t as follows:

$$m_t = \frac{S_t}{\frac{1}{T} \sum_{j=1}^T S_j} \quad (2)$$

$$\epsilon_t = \epsilon_{\text{high}} \cdot m_t \quad (3)$$

Implementation Note: A lower bound δ can be applied ($m_t = \max(m_t, \delta)$) to prevent ϵ_t from collapsing entirely to zero on extreme filler tokens.

3.3 Step 3: The AD-CISPO Objective Update

With the frozen tensor ϵ_t successfully derived from π_{ref} , it is passed to the active policy π_θ . The standard scalar is replaced, yielding the final Adaptive and Dense CISPO objective:

$$L_{\text{AD-CISPO}}(\theta) = -\mathbb{E} \left[\text{detach}(\min(r_t(\theta), \epsilon_t)) \cdot \hat{A}_t \cdot \log \pi_\theta(a_t | s_t) \right] \quad (4)$$

Result: The active policy seamlessly executes gradient updates where critical tokens possess widened learning corridors, while filler tokens are aggressively clamped. The pipeline strictly maintains standard training throughput by eliminating all $O(N^2)$ tracking requirements.